MODULE 30 · DOMAIN 5 - IMPLEMENT INFORMATION EXTRACTION SOLUTIONS

Enrichment with skills

Skillsets and the enrichment tree, context and mappings, built-in skills for text, images and layout, custom skills, integrated vectorization, and caching.

DOMAIN 5 - Information extraction WEIGHT 10-15% MODULE 30 of 39

admin@sthithapragna.com

A skillset is an **assembly line**. Each skill adds a part to the document, and only what you **map** reaches the index.

admin@sthithapragna.com

What a skillset is made of

- An **array of skills**: built-in from Microsoft, or **custom** ones you host.
- Each skill has a **context**, **inputs** (by source path) and **outputs**.
- Paths start at **/document**, the root of the enrichment tree built during document cracking.
- Skill outputs become **new nodes**, always added under the skill's context node.
- Nodes are **immutable**. A later skill adds new nodes rather than editing earlier ones.

> ON THE EXAM
>
> Skills run in parallel unless one skill's output is another's input, which chains them in order.

Context decides how often a skill runs. **/document** runs once per file; **/document/pages/*** runs once per chunk.

One skill, running once per chunk

```
{
  "@odata.type": "#Microsoft.Skills.Text.V3.EntityRecognitionSkill",
  "context": "/document/pages/*",
  "categories": ["Organization"],
  "inputs": [
    {"name": "text", "source": "/document/pages/*"}],
  "outputs": [
    {"name": "organizations", "targetName": "chunk_orgs"}]
}
```

ON THE EXAM

Outputs land at /document/pages/*/chunk_orgs, one set per chunk, ready for an output field mapping.

Built-in skills, by job

JOB	SKILLS
Split and shape text	Text Split, Text Merge, Shaper, Conditional
Understand text	Entity Recognition, PII Detection, Language Detection, Translation, Key Phrases
Read and describe images	OCR, Image Analysis, GenAI Prompt (verbalization)
Layout and extraction	Content Understanding, Document Layout, Document Extraction
Create vectors	Azure OpenAI Embedding, Azure Vision multimodal embeddings, AML
Anything else	Custom Web API skill

ON THE EXAM

For new skillsets, Microsoft recommends the Content Understanding skill over Document Layout.

Document Layout or Content Understanding?

DOCUMENT LAYOUT SKILL

Still supported

- Chunks by document structure
- Supported for existing pipelines
- Familiar in older samples

CONTENT UNDERSTANDING SKILL

Recommended for new work

- Content extraction and chunking in one skill
- Semantic chunking
- AI-generated image descriptions
- Cross-page table extraction

ON THE EXAM

Same advice as module 23: Document Layout keeps working, but new skillsets should use Content Understanding.

admin@sthithapragna.com

Two kinds of mapping

- **Field mappings** carry **raw source fields** into index fields.
- **Output field mappings** carry **enriched nodes** into index fields.
- Unmapped enrichments are **discarded** when the run ends.
- A **knowledge store** can persist enriched output to Azure Storage for reuse or analysis.
- If you project to a knowledge store, add a **Shaper skill** as the last step.

> THE SIGNAL
>
> 'The skill ran but the field is empty' is almost always a missing or mistyped output field mapping.

Two halves of the same feature

- An **embedding skill** vectorizes content during **indexing**.
- A **vectorizer**, defined in the index and referenced by a **vector profile**, embeds the **query**.
- Options include Azure OpenAI, Azure Vision multimodal, AML, and a custom Web API.
- Both halves must use the **same model and dimensions**, as module 13 showed.
- Without a vectorizer, the application must embed every query itself before searching.

> ON THE EXAM
>
> A custom Web API vectorizer handles one record at a time, and its errors aren't visible for query-time debugging.

The skill embeds the documents. The vectorizer embeds the question. Configure both, or your app must do half the job.

When you need your own skill

- Logic Microsoft doesn't provide: internal lookups, bespoke parsing, your own classifier.
- The **custom Web API skill** calls your HTTPS endpoint during indexing.
- The **AML skill** calls a machine learning endpoint or a catalogue model.
- Custom skills run at **indexing time only**, not at query time.
- Authenticate with a **managed identity**, setting authResourceId to the API's audience.

> ON THE EXAM
>
> Enriching every document at ingestion costs once. Doing the same work per query costs on every search.

What a custom skill must return

- Requests arrive as a **values** array, each record with a **recordId** and **data**.
- Every response record needs its **recordId**, plus **errors** and **warnings** arrays.
- Errors and warnings are **required fields**, but their value may be **null**.
- Records with an unknown, missing or duplicated recordId are **discarded**: no enrichment.
- The indexer **retries twice** on 429, 502 and 503. Default timeout is 30 seconds.

> THE SIGNAL
>
> recordId is how the service correlates your answers with its documents. Without it, output goes nowhere.

A valid custom skill response

```
{
  "values": [
    {"recordId": "a1",
     "data": {"contractDate": "2026-11-03"},
     "errors": null,
     "warnings": null},
    {"recordId": "c3",
     "data": {},
     "errors": [{"message": "No date found in contract text"}],
     "warnings": null}
  ]
}
```

ON THE EXAM

Record c3 failed cleanly: it still returns its recordId, with the reason in errors, which appears in indexer history.

Incremental enrichment and the cache

- The **enrichment cache** stores document cracking output and every skill's output per document.
- On later runs, only **new or changed skills and documents** incur AI processing charges.
- Caching uses Azure Storage, which is billable, but usually far cheaper than reprocessing.
- Changing a skill invalidates it **and every skill downstream of it**.
- Changing output field mappings re-projects documents, without re-running the skills.

> ON THE EXAM
>
> Adding one skill to a million-document pipeline is the case the cache exists for. It's still a preview feature.

Enrichment symptoms and causes

SYMPTOM	LIKELY CAUSE
Skill ran, index field empty	Missing or wrong output field mapping
Enrichment applied once per file, not per chunk	Context set to /document
Custom skill output ignored	Missing recordId, errors or warnings in the response
Vector queries need app-side embedding	No vectorizer on the index
Every edit reprocesses everything	Enrichment cache not enabled
Some documents silently lack fields	Per-document errors in indexer history

ON THE EXAM

Debug sessions in the portal show the enriched document node by node, which settles most path questions.

> A team writes a custom Web API skill that looks up supplier risk ratings. It returns HTTP 200, with an array of objects containing only a data property.
>
> The indexer reports no errors, but the risk rating field is empty for every document.

What should the team change?

A. Return each record's recordId, with errors and warnings arrays.

B. Raise the skill's timeout above the default of 30 seconds.

C. Replace the custom skill with an AML skill calling the same logic.

D. Change the skill's HTTP method from POST to PUT in the definition.

◆ PAUSE AND DECIDE

What should the team change?

A Return recordId, errors, warnings.

Without recordId, responses can't be correlated, so the enrichment is discarded.

B Longer timeout.

The endpoint answered with 200. Nothing timed out.

C AML skill instead.

A different skill type with the same contract problem.

D PUT instead of POST.

Both methods are allowed, and neither fixes the response shape.

THE SIGNAL

Successful call, no errors, empty field: the response didn't follow the custom skill contract.

> A skillset splits long reports into pages, then runs entity recognition to tag organisations. The organisations field holds entities for the whole report, not per chunk.
>
> Results should let users filter chunks by the organisation each one mentions.

What should the team change?

A Add a second entity recognition skill after the first one in the skillset.

B Change the index field for organisations to a collection of strings.

C Set the skill's context to the pages collection, so it runs per chunk.

D Project the enriched documents into a knowledge store and query that.

◆ PAUSE AND DECIDE

What should the team change?

A. A second skill.
Another skill with the same context repeats the same document-level result.

B. Change the field type.
Storage shape, not scope. The entities are still computed per document.

C. Context on the pages collection.
The skill then runs once per chunk, with outputs under each chunk node.

D. Knowledge store.
Persists enrichments elsewhere; it doesn't change how they're computed.

THE SIGNAL

Per-file results when you wanted per-chunk results: context is the property that decides.

> An index has a vector field populated during indexing by the Azure OpenAI embedding skill. Vector search works from a test app that generates embeddings itself.
>
> A new team wants to query the index with plain text from a low-code tool that can't create embeddings.

What should the team configure?

A Re-run the indexer so the vector field is rebuilt with newer content.

B A vectorizer on the index, referenced from the vector profile.

C Switch the new team's queries to keyword search over the text fields.

D Export the embeddings to Blob Storage so the tool can look them up.

◆ PAUSE AND DECIDE

What should the team configure?

A Re-run the indexer.
Re-embeds documents. Queries still arrive as plain text.

B A vectorizer on the index.
The service embeds query text at query time, using the same model.

C Keyword search only.
Abandons vector search rather than enabling it for the new tool.

D Export embeddings.
Document vectors don't help embed an incoming question.

THE SIGNAL

Documents embedded, queries not: the missing half of integrated vectorization is the vectorizer.

> A pipeline enriches two million documents with OCR, translation and embeddings. The team now wants to add PII detection as one more skill.
>
> A full re-run would take days and repeat every OCR and embedding charge already paid.

What should the team do?

A Reduce the indexer's batch size so the re-run costs less overall.

B Delete the index and rebuild it from scratch with the new skillset.

C Split the skillset across two indexers that run on alternate nights.

D Enable the enrichment cache, so only the new skill is processed.

◆ PAUSE AND DECIDE

What should the team do?

A Smaller batches.
Changes pacing, not how much AI processing is repeated.

B Rebuild from scratch.
Guarantees paying for every skill again, on every document.

C Two indexers.
Splits the work in time, while still reprocessing everything.

D Enrichment cache.
Cached cracking and skill outputs are reused; only the new skill incurs charges.

THE TIEBREAKER

The cache exists for exactly this: adding or editing a skill without reprocessing unchanged work.

Enrichment in five lines

- **Skills add nodes to an enrichment tree; only mapped nodes reach the index.**
- **Context decides how often a skill runs**, and where its output lands.
- **Custom skills must return recordId, errors and warnings**, or output is discarded.
- **Embedding skill for documents; vectorizer for queries.** Configure both.
- **The enrichment cache** stops one new skill costing a full re-run.

> ON THE EXAM
>
> Next, module 31: semantic, hybrid and vector search, and connecting retrieval to agents.